

ROADMAP 2.0 · EXPANSÃO SOBRE SISTEMA JÁ EXISTENTE

MedFlow 2.0

Controle de Acesso Clínico Real

Evolução do MedFlow para o comportamento de uma clínica de verdade: múltiplas especialidades, equipe de apoio (triagem, recepção, faturamento), e controle de acesso granular — cada papel vê exatamente o que deveria ver, nada mais.

RBAC granular

Multi-especialidade

Triagem & Recepção

Prontuário protegido

Cirurgias & Prescrições

Auditoria

Spring Method Security



Este documento é aditivo — não deve quebrar nada do MedFlow 1.0 já implementado

Sumário

VISÃO GERAL

O que já existe e não pode quebrar	03
Visão de negócio: como uma clínica real funciona	04
Matriz de permissões (RBAC) — o coração deste roadmap	05
Modelo de dados expandido	07

FASES DE EXECUÇÃO

Fase 2.1 — Papéis (roles) e migração segura	09
Fase 2.2 — Múltiplas especialidades por médico	11
Fase 2.3 — Separação de dados cadastrais x dados clínicos	12
Fase 2.4 — Segurança granular (Method Security)	14
Fase 2.5 — Módulo de Triagem	16
Fase 2.6 — Módulo de Recepção	18
Fase 2.7 — Módulo clínico do médico (prontuário, prescrições, cirurgias)	19
Fase 2.8 — Módulo de Faturamento	22
Fase 2.9 — Auditoria (quem fez o quê, e quando)	23
Fase 2.10 — Frontend: páginas e rotas por papel	24
Fase 2.11 — Testes de segurança e regras de negócio	26
Fase 2.12 — Documentação OpenAPI com segurança	27

ANEXOS

Anexo A — Prompt pronto para entregar à IA	28
Anexo B — Checklist geral de entrega	30
Anexo C — Protocolo de testes rigorosos obrigatório	31

VISÃO GERAL

O que já existe e não pode quebrar

Este roadmap é uma expansão, não uma reescrita. Toda decisão abaixo assume que o MedFlow 1.0 já está rodando.

Antes de qualquer código novo, o mais importante deste documento é uma regra: **tudo aqui é aditivo**. Nenhuma fase deste roadmap deve remover, renomear ou alterar o comportamento de algo que já funciona hoje, a menos que explicitamente indicado (e, nesses casos, sempre via **migration versionada**, nunca alteração manual no banco).

Inventário do que já está implementado

Camada	O que existe hoje
Backend	Java 21, Spring Boot, Spring Security + JWT, JPA/Hibernate, Flyway, PostgreSQL, RabbitMQ. Entidades: <code>Usuario</code> , <code>Medico</code> , <code>Paciente</code> , <code>Especialidade</code> , <code>Consulta</code> , <code>Prontuario</code> , <code>Exame</code> , <code>Pagamento</code> .
Autenticação	<code>POST /auth/register</code> , <code>POST /auth/login</code> , JWT com claims <code>{sub, role, iat, exp}</code> . Roles atuais: <code>PACIENTE</code> , <code>MEDICO</code> , <code>ADMIN</code> .
Frontend	React + Vite + TypeScript + Tailwind + React Router + TanStack Query. <code>AuthContext</code> decodifica o JWT e expõe o role atual. <code>Layout</code> já renderiza menu condicional por role.
Infra	Docker, Docker Compose, Nginx (frontend), manifests Kubernetes, CI/CD no GitHub Actions.

✓ Compromissos de compatibilidade deste roadmap

- Usuários já cadastrados com role `PACIENTE`, `MEDICO` ou `ADMIN` continuam funcionando exatamente como hoje, sem precisar recriar conta.
- Toda mudança de schema é feita via nova migration Flyway (`V2_...`, `V3_...`), nunca editando uma migration antiga já aplicada.
- Endpoints existentes mantêm sua assinatura (rota, verbo, formato de resposta) — o que muda é **quem tem permissão** de chamá-los, não o contrato em si, exceto onde uma nova fase disser explicitamente o contrário (ex.: separar dados clínicos de dados cadastrais do paciente, Fase 2.3).
- O claim `role` do JWT deixa de ser um valor único simples e passa a ser tratado como *autoridade* dentro de um conjunto maior de papéis — isso é uma expansão do enum, não uma substituição.

Visão de negócio: como uma clínica real funciona

Antes das tabelas técnicas, entenda o fluxo humano que o sistema precisa espelhar.

Numa clínica de verdade, um paciente não fala direto com o médico. Existe um fluxo, com pessoas diferentes cuidando de partes diferentes — e cada uma delas **não deveria conseguir ver tudo**, por questão de privacidade, segurança e até responsabilidade legal (dados de saúde são dados sensíveis, protegidos por lei).

O fluxo típico de um atendimento

1. **Recepção:** o paciente chega (ou liga), a recepcionista cadastra/confirma seus dados pessoais e agenda a consulta. Ela vê nome, telefone, endereço, convênio — mas nunca o histórico médico.
2. **Triagem:** antes da consulta, um profissional de triagem mede sinais vitais (pressão, temperatura, peso) e registra a queixa principal, definindo uma prioridade de atendimento. Ele vê os dados básicos do paciente e o que ele mesmo registra — mas não o prontuário médico completo de consultas anteriores.
3. **Consulta médica:** o médico atende, acessa o prontuário completo (histórico, prescrições anteriores, exames), registra a evolução do atendimento atual, pode prescrever medicamentos, solicitar exames ou marcar uma cirurgia.
4. **Faturamento:** depois da consulta, o setor financeiro processa o pagamento — vê valores e status de cobrança, mas não o conteúdo clínico do atendimento.
5. **Administração:** por cima de tudo, o admin gerencia usuários, especialidades, permissões e pode ver relatórios agregados — mas isso não deveria significar acesso irrestrito ao prontuário de cada paciente sem motivo, embora, para simplificar o escopo deste projeto de estudo, o admin terá acesso total (numa clínica real, mesmo isso costuma ser auditado e restrito).



A ideia central deste roadmap 2.0

O MedFlow 1.0 tratava "usuário logado" quase como uma coisa binária: autenticado ou não. O MedFlow 2.0 introduz o conceito de que **estar autenticado não significa ter acesso a tudo** — cada papel enxerga uma fatia diferente do mesmo sistema, e o backend é quem garante isso, não apenas o frontend escondendo botões.



Erro comum a evitar

Esconder um menu no frontend para um papel **não é segurança** — é só experiência de uso. Toda regra de "quem pode ver o quê" deste roadmap precisa ser garantida no **backend** (Spring Security), porque qualquer pessoa pode chamar a API diretamente, sem passar pelo frontend.

Matriz de permissões (RBAC)

A tabela mais importante deste documento — toda fase de segurança implementa exatamente isto.

Papéis (roles) do MedFlow 2.0

Role	Quem é
ADMIN	Gestão do sistema: usuários, especialidades, permissões, relatórios (já existe)
MEDICO	Atendimento clínico completo dos seus pacientes (já existe)
PACIENTE	Usuário final, vê apenas seus próprios dados (já existe)
RECEPCIONISTA (novo)	Cadastro de pacientes e agendamento — sem acesso clínico
TRIAGEM (novo)	Registro de sinais vitais e prioridade de atendimento — acesso clínico limitado
FATURAMENTO (novo)	Gestão financeira de pagamentos — sem acesso clínico

Matriz de acesso por recurso

Legenda: **SIM** = acesso total · **PARCIAL** = acesso restrito/próprio · **NÃO** = sem acesso.

Recurso	ADMIN	MEDICO	RECEPCIONISTA	TRIAGEM	FATURAMENTO	PACIENTE
Dados cadastrais do paciente (nome, telefone, endereço)	SIM	SIM	SIM	SIM	PARCIAL	Próprio
Agendar/remarcar consultas	SIM	Próprias	SIM	NÃO	NÃO	Próprias
Registrar triagem (sinais vitais, queixa, prioridade)	SIM	SIM	NÃO	SIM	NÃO	NÃO
Ver prontuário clínico completo	SIM	Seus pacientes	NÃO	NÃO	NÃO	Próprio
Editar prontuário / registrar evolução	NÃO	Seus pacientes	NÃO	NÃO	NÃO	NÃO
Prescrever medicamentos	NÃO	Seus pacientes	NÃO	NÃO	NÃO	NÃO
Marcar/gerenciar cirurgias	SIM	Suas cirurgias	NÃO	NÃO	NÃO	NÃO
Ver/gerenciar pagamentos	SIM	NÃO	Visualizar	NÃO	SIM	Próprios
Gerenciar especialidades	SIM	NÃO	NÃO	NÃO	NÃO	NÃO
Gerenciar usuários e papéis	SIM	NÃO	NÃO	NÃO	NÃO	NÃO
Ver logs de auditoria	SIM	NÃO	NÃO	NÃO	NÃO	NÃO

 **Como ler "PARCIAL"**

"Próprias"/"Seus pacientes" significa que a regra não é só "tem o role X", mas também "é dono/responsável por este registro específico". Ex.: um médico só edita o prontuário de um paciente que ele efetivamente atendeu/está atendendo — não o de qualquer paciente do sistema. Essa checagem é feita a nível de *registro*, não só de *rota* (ver Fase 2.4).

Modelo de dados expandido

Novas entidades e como elas se conectam ao que já existe, sem quebrar nada.

Novas entidades

Entidade	Campos principais	Relacionamentos
Funcionario	id, usuario_id, cargo (RECEPCIONISTA/TRIAGEM/FATURAMENTO), data_admissao	1:1 com Usuario
MedicoEspecialidade	medico_id, especialidade_id	Tabela associativa N:N entre Medico e Especialidade (substitui o vínculo 1:N único anterior, ver Fase 2.2)
Triagem	id, paciente_id, consulta_id, realizado_por_id, pressao_arterial, temperatura, peso, altura, queixa_principal, prioridade (BAIXA/MEDIA/ALTA/EMERGENCIA), data_hora	N:1 com Paciente, N:1 com Consulta, N:1 com Funcionario
Prescricao	id, prontuario_id, medico_id, medicamento, dosagem, instrucoes, data	N:1 com Prontuario
Cirurgia	id, paciente_id, medico_id, tipo, data_hora_prevista, local, status (AGENDADA/REALIZADA/CANCELADA), observacoes	N:1 com Paciente, N:1 com Medico
AuditLog	id, usuario_id, entidade, entidade_id, acao (CREATE/UPDATE/DELETE/VIEW), timestamp, detalhes (JSON)	N:1 com Usuario

Entidades existentes que ganham ajustes (via migration, sem quebrar dados)

Entidade	Ajuste	Impacto na compatibilidade
Usuario	Coluna <code>role</code> passa a aceitar os novos valores do enum expandido	Registros existentes com <code>PACIENTE</code> / <code>MEDICO</code> / <code>ADMIN</code> continuam válidos, nenhum dado é alterado
Medico	Relacionamento com especialidade migra de coluna única <code>especialidade_id</code> para tabela associativa <code>MedicoEspecialidade</code>	Migration copia o vínculo único existente para a nova tabela antes de remover a coluna antiga — nenhum médico perde sua especialidade atual
Paciente	Nenhuma alteração estrutural — a separação cadastral/ clínica (Fase 2.3) é feita a nível de DTO e permissão , não de tabela	Zero impacto no schema existente

✓

Estratégia de migration segura para o vínculo médico-especialidade

1) Criar a nova tabela `medico_especialidade` . 2) Migration de dados: `INSERT INTO medico_especialidade (medico_id, especialidade_id) SELECT id, especialidade_id FROM medico WHERE especialidade_id IS NOT NULL;` 3) Só depois, numa migration separada, remover a coluna antiga `especialidade_id` de `medico` . Nunca fazer isso em uma única migration — se algo falhar no meio, você quer poder identificar exatamente em qual etapa.

2.1 Papéis (roles) e migração segura

Expandir o enum de roles sem invalidar nenhum usuário já cadastrado.

Flyway

Enum Role

Spring Security Authorities

O que fazer

- Expandir o enum `Role` no backend: `ADMIN, MEDICO, PACIENTE, RECEPCIONISTA, TRIAGEM, FATURAMENTO`.
- Criar migration `V__add_novos_roles.sql` que, se o banco usa um `CHECK constraint` ou tipo `ENUM` nativo do Postgres na coluna `role`, o expande para incluir os novos valores (nunca recriando a coluna do zero).
- Criar a entidade `Funcionario` e sua migration, vinculada 1:1 a `Usuario` para os papéis `RECEPCIONISTA`, `TRIAGEM` e `FATURAMENTO` (espelhando o padrão que já existe entre `Usuario` e `Medico / Paciente`).
- Atualizar o fluxo de `POST /auth/register` (ou criar `POST /admin/funcionarios` restrito a ADMIN) para permitir criar usuários com os novos papéis — na prática, numa clínica real, funcionários não se auto-registram como pacientes fazem; apenas o ADMIN cria essas contas.

```
// Role.java – expansão do enum existente
public enum Role {
    ADMIN, MEDICO, PACIENTE,
    RECEPCIONISTA, TRIAGEM, FATURAMENTO // novos
}
```

⚠ Cuidado com o registro público

O endpoint `POST /auth/register` hoje provavelmente aceita role vinda do próprio request. Isso precisa mudar: o **registro público (sem autenticação) só pode criar usuários com role `PACIENTE`**. Qualquer outro papel (MEDICO, RECEPCIONISTA, TRIAGEM, FATURAMENTO, ADMIN) só pode ser criado por um ADMIN já autenticado, via endpoint separado e protegido. Isso não quebra o fluxo de paciente existente, mas fecha uma brecha real de segurança.

✅ Definição de pronto (DoD) desta fase

- Todos os usuários existentes continuam logando normalmente, sem nenhuma alteração de comportamento
- `POST /auth/register` aberto só cria pacientes, mesmo que o corpo da requisição tente forçar outro role
- Um ADMIN autenticado consegue criar um usuário RECEPCIONISTA, TRIAGEM ou FATURAMENTO via endpoint próprio
- O JWT emitido para esses novos papéis contém o claim `role` corretamente

2.2 Múltiplas especialidades por médico

Um médico real costuma ter mais de uma especialidade — corrigir isso no modelo.

O que fazer

- Criar a tabela associativa `medico_especialidade` (N:N entre `Medico` e `Especialidade`), seguindo a estratégia de migration segura descrita no capítulo "Modelo de dados expandido".
- Atualizar `MedicoDTO` para retornar uma **lista** de especialidades em vez de uma única.
- Atualizar o endpoint `GET /medicos?especialidade=X` para funcionar com o novo relacionamento N:N.
- Atualizar o formulário de cadastro/edição de médico (ADMIN) no frontend para permitir selecionar múltiplas especialidades (multi-select).

✓ Definição de pronto (DoD) desta fase

- Médicos já cadastrados mantêm sua especialidade original, agora migrada para a tabela associativa
- É possível associar um médico a duas ou mais especialidades via API/admin
- A busca de médicos por especialidade continua funcionando para o frontend de paciente (Fase 1)

2.3 Separação de dados cadastrais x dados clínicos

A base técnica que torna a matriz de permissões possível de verdade.

Hoje, provavelmente, um único `PacienteDTO` devolve tudo junto: nome, telefone, e talvez referências ao prontuário. Para respeitar a matriz de permissões (recepção vê dados pessoais, mas não vê prontuário), é preciso separar isso em **dois DTOs diferentes**, com endpoints/permissões diferentes — sem alterar a entidade `Paciente` no banco.

O que fazer

- Criar `PacienteResumoDTO`: nome, telefone, email, endereço, convênio, data de nascimento — **nada clínico**. Usado por RECEPCIONISTA e TRIAGEM.
- Criar `PacienteClinicoDTO`: inclui o resumo acima + referência ao prontuário, histórico de consultas, alergias, condições pré-existentes. Usado apenas por MEDICO (dos seus pacientes) e ADMIN.
- O endpoint `GET /pacientes/{id}` passa a devolver um DTO ou outro **dependendo do role de quem está chamando** (a mesma rota, resposta diferente por permissão) — ou, alternativa mais explícita, dividir em duas rotas: `GET /pacientes/{id}/resumo` e `GET /pacientes/{id}/clinico`.
- Atualizar o mapper correspondente para nunca vazar campos clínicos dentro do DTO de resumo, nem por engano.

💡 Recomendação de implementação

Prefira **duas rotas explícitas** (`/resumo` e `/clinico`) em vez de uma rota "inteligente" que muda de resposta por role. Fica mais fácil de testar, documentar no Swagger, e de auditar quem acessou dado clínico versus quem acessou só dado cadastral (isso alimenta a Fase 2.9, de auditoria).

✅ Definição de pronto (DoD) desta fase

- Uma chamada autenticada como RECEPCIONISTA a qualquer rota de dado clínico retorna `403 Forbidden`
- `PacienteResumoDTO` nunca contém, nem por acidente, campos como diagnóstico, prescrição ou histórico de prontuário
- O frontend de paciente (tela de perfil do próprio paciente, Fase 1) continua funcionando normalmente

2.4 Segurança granular (Method Security)

De "autenticado ou não" para "este usuário específico pode fazer isto neste registro específico".

@EnableMethodSecurity

@PreAuthorize

SecurityService (bean custom)

O que fazer

- Habilitar `@EnableMethodSecurity` na configuração do Spring Security (se ainda não estiver habilitado).
- Criar um bean `SecurityService` com métodos reutilizáveis de checagem "é dono do registro", por exemplo:
 - `isOwnerPaciente(Authentication auth, Long pacienteId)` — true se o usuário autenticado é o próprio paciente
 - `isMedicoResponsavel(Authentication auth, Long pacienteId)` — true se o médico autenticado já atendeu/está agendado com esse paciente
- Aplicar `@PreAuthorize` em cada endpoint sensível, combinando role + posse do registro, conforme a matriz de permissões.
- Padronizar respostas: falha de autenticação → `401`; usuário autenticado sem permissão → `403`, sempre com corpo de erro consistente com o handler global já existente (Fase 2 do roadmap 1.0).

```
@PreAuthorize("hasRole('MEDICO') and @securityService.isMedicoResponsavel(authentication, #pacienteId)")
@GetMapping("/pacientes/{pacienteId}/clinico")
public PacienteClinicoDTO buscarDadosClinicos(@PathVariable Long pacienteId) { ... }

@PreAuthorize("hasAnyRole('RECEPCIONISTA', 'TRIAGEM', 'MEDICO', 'ADMIN')")
@GetMapping("/pacientes/{pacienteId}/resumo")
public PacienteResumoDTO buscarResumo(@PathVariable Long pacienteId) { ... }
```

Definição — por que "posse do registro" e não só role

Um `@PreAuthorize(hasRole('MEDICO'))` sozinho garante só que a pessoa é médico — não que ela é o médico *certo* para aquele paciente específico. Sem a checagem de posse (`isMedicoResponsavel`), qualquer médico do sistema poderia ver o prontuário de qualquer paciente, o que não reflete uma clínica real.

✓ Definição de pronto (DoD) desta fase

- Toda rota da matriz de permissões tem sua regra de `@PreAuthorize` correspondente implementada
- Um médico que nunca atendeu determinado paciente recebe `403` ao tentar acessar o prontuário desse paciente
- Um paciente só acessa seus próprios dados, nunca os de outro paciente, mesmo trocando o ID na URL manualmente

2.5 Módulo de Triage

O primeiro contato clínico do paciente — mas com acesso limitado de propósito.

O que fazer

- Entidade e migration de `Triage` (conforme "Modelo de dados expandido").
- `POST /triagens` : registra uma triagem vinculada a uma consulta e paciente. Só `TRIAGEM` ou `MEDICO / ADMIN` podem criar.
- `GET /triagens/consulta/{consultaId}` : consulta o registro de triagem de uma consulta específica.
- Campo `prioridade` (enum `BAIXA/MEDIA/ALTA/EMERGENCIA`) deve refletir numa ordenação especial na fila de atendimento do médico (ex.: `GET /medicos/agenda-hoje` ordena por prioridade de triagem, não só por horário agendado).

✓ Definição de pronto (DoD) desta fase

- Um usuário TRIAGEM consegue registrar sinais vitais e prioridade para uma consulta do dia
- Um usuário TRIAGEM recebe `403` ao tentar acessar o prontuário clínico completo do mesmo paciente
- O médico vê a prioridade de triagem refletida na sua agenda do dia

2.6 Módulo de Recepção

Cadastro e agendamento — a porta de entrada da clínica.

O que fazer

- `POST /pacientes` (cadastro completo de um novo paciente, incluindo criação do `Usuario` vinculado) liberado para RECEPCIONISTA e ADMIN, além do próprio fluxo de auto-registro do paciente já existente.
- RECEPCIONISTA ganha acesso a `POST /consultas` e `PUT /consultas/{id}/remarcar` em nome de qualquer paciente (diferente do paciente, que só agenda para si mesmo) — reaproveitando as regras de conflito de horário já implementadas na Fase 7 do roadmap 1.0, sem duplicar essa lógica.
- Endpoint `GET /consultas/hoje` : agenda do dia para a recepção acompanhar check-ins, com status combinando confirmação da consulta e da triagem.
- Endpoint `PUT /consultas/{id}/checkin` : marca que o paciente chegou fisicamente à clínica.

💡 Reuso, não duplicação

A regra de "não permitir conflito de horário para o mesmo médico" já existe no `ConsultaService` desde o roadmap 1.0. A recepção deve chamar o **mesmo service**, só que autorizado a informar o `pacienteId` em vez de assumir o usuário logado — não recriar essa lógica em outro lugar.

✅ Definição de pronto (DoD) desta fase

- Uma recepcionista consegue cadastrar um novo paciente e agendar sua primeira consulta sem que o paciente precise se auto-cadastrar
- Uma recepcionista recebe `403` ao tentar acessar qualquer rota de prontuário/prescrição/cirurgia
- Check-in reflete corretamente no status visível para a triagem e para o médico

2.7 Módulo clínico do médico

Prontuário, prescrições e cirurgias — o núcleo do trabalho médico.

Prontuário e evolução

- O módulo de **Prontuario** já existe (roadmap 1.0, Fase 9) — nesta fase, ele passa a ser protegido pelas regras de **@PreAuthorize** da Fase 2.4 (só o médico responsável ou ADMIN acessa/edita).
- Adicionar campo/registro de **evolução por consulta**: cada atendimento gera uma nova entrada de evolução no prontuário (não sobrescreve o histórico anterior) — isso pode ser uma nova entidade **EvolucaoClinica** (id, prontuario_id, consulta_id, texto, data) se ainda não existir algo equivalente.

Prescrições

- Entidade **Prescricao** (conforme "Modelo de dados expandido").
- **POST /prescricoes** : só MEDICO (dono do atendimento) pode criar, vinculada a um prontuário/consulta.
- **GET /prescricoes/paciente/{id}** : histórico de prescrições — visível ao próprio paciente (somente leitura) e ao médico responsável.

Cirurgias

- Entidade **Cirurgia** (conforme "Modelo de dados expandido").
- **POST /cirurgias** : médico marca uma cirurgia para um paciente seu, com tipo, data prevista e local.
- **PUT /cirurgias/{id}/status** : atualiza o ciclo **AGENDADA → REALIZADA** ou **AGENDADA → CANCELADA**, seguindo o mesmo padrão de máquina de estados já usado em **Consulta** no roadmap 1.0.
- Ao marcar uma cirurgia, gerar automaticamente um **Pagamento** vinculado (reaproveitando o módulo de pagamentos da Fase 8 do roadmap 1.0), com um novo campo **origem** (**CONSULTA** ou **CIRURGIA**) na entidade **Pagamento** para diferenciar a cobrança.

✓ Definição de pronto (DoD) desta fase

- O médico consegue registrar uma nova evolução clínica a cada consulta, sem apagar o histórico anterior
- O médico consegue prescrever medicamentos e marcar cirurgias apenas para pacientes que ele efetivamente atende
- O paciente vê suas próprias prescrições e cirurgias, em modo somente leitura
- Uma cirurgia marcada gera automaticamente um pagamento pendente vinculado

2.8 Módulo de Faturamento

Quem cuida do dinheiro não precisa (nem deve) ver o motivo clínico por trás dele.

O que fazer

- Liberar para o role **FATURAMENTO** os endpoints já existentes de **Pagamento** (Fase 8 do roadmap 1.0): listar, confirmar pagamento, estornar.
- Garantir que o **PagamentoDTO** devolvido não exponha detalhes clínicos da consulta/cirurgia associada — apenas identificadores, valor, status e o campo **origem** criado na fase anterior.
- Endpoint **GET /pagamentos/relatorio?inicio=...&fim=...**: relatório simples de faturamento por período, liberado para FATURAMENTO e ADMIN.

✓ Definição de pronto (DoD) desta fase

- Um usuário FATURAMENTO consegue confirmar/estornar pagamentos e ver relatórios financeiros
- Um usuário FATURAMENTO recebe **403** ao tentar acessar qualquer rota clínica (prontuário, prescrição, triagem, cirurgia)

2.9 Auditoria

Registrar quem acessou/alterou o quê — essencial em qualquer sistema que lide com dados de saúde.

AOP (Aspect)

@EntityListeners

AuditLog

O que fazer

- Entidade `AuditLog` (conforme "Modelo de dados expandido").
- Criar um **aspecto (AOP)** ou interceptor que registra automaticamente toda chamada a endpoints marcados como sensíveis (prontuário, prescrição, cirurgia, dados clínicos do paciente): quem chamou, quando, em qual registro, e qual ação.
- Para alterações de dados (create/update/delete) nas entidades clínicas, usar `@EntityListeners` do JPA para capturar automaticamente, sem precisar espalhar código de log manualmente em cada service.
- Endpoint `GET /admin/auditoria?entidade=...&usuarioId=...`, restrito a ADMIN, para consulta dos logs.

💡 Por que isso não é exagero

Numa clínica real, saber *quem visualizou o prontuário de qual paciente e quando* não é luxo — é exigência legal em praticamente qualquer país (no Brasil, alinhado à LGPD para dados sensíveis de saúde). Treinar esse hábito agora, mesmo num projeto de estudo, é um diferencial real de quem entende sistemas de produção.

✓ Definição de pronto (DoD) desta fase

- Toda visualização e alteração de dado clínico gera uma entrada em `AuditLog`, automaticamente
- O ADMIN consegue filtrar os logs por usuário e por entidade
- Nenhum outro papel além de ADMIN acessa a rota de auditoria

2.10 Frontend: páginas e rotas por papel

Espelhando no frontend tudo o que o backend agora garante de verdade.

Novas páginas

Página	Role	Descrição
/recepcao	RECEPCIONISTA	Cadastro de pacientes, agenda do dia, check-in
/triagem	TRIAGEM	Fila de pacientes aguardando triagem, formulário de sinais vitais e prioridade
/faturamento	FATURAMENTO	Lista de pagamentos, confirmação/estorno, relatório por período
/prontuario/:pacienteId	MEDICO / ADMIN	Visão completa do histórico clínico, formulário de evolução, prescrição e cirurgia
/admin/funcionarios	ADMIN	CRUD de funcionários (recepção, triagem, faturamento) e seus papéis
/admin/auditoria	ADMIN	Consulta de logs de auditoria

O que fazer

- Estender o componente `ProtectedRoute` existente para aceitar uma lista de `allowedRoles`, redirecionando (ou mostrando "acesso negado") quando o role atual não está na lista.
- Atualizar o `Layout` /menu de navegação para incluir os novos itens, condicionados ao role — reaproveitando o padrão já usado para PACIENTE/MEDICO/ADMIN.
- Tratar globalmente respostas `403` da API (ex.: um interceptor do Axios/TanStack Query) para exibir uma mensagem amigável de "sem permissão", em vez de quebrar a tela.

✓ Definição de pronto (DoD) desta fase

- Cada novo papel, ao logar, vê apenas os menus e páginas correspondentes à sua função
- Tentar acessar manualmente uma URL fora do próprio papel (ex.: recepcionista digitando `/prontuario/1` na barra de endereço) mostra uma tela de acesso negado, sem quebrar a aplicação
- As telas já existentes de PACIENTE, MEDICO e ADMIN (roadmap 1.0) continuam funcionando sem regressão

2.11 Testes de segurança e regras de negócio

Provar, com testes automatizados, que a matriz de permissões é respeitada de verdade.

JUnit 5

MockMvc

Testcontainers

O que fazer

- Para cada linha da matriz de permissões, escrever ao menos um teste de **acesso negado** (o papel sem permissão recebe `403`) e um teste de **acesso permitido** (o papel correto consegue realizar a ação).
- Teste específico de "posse do registro": um médico A não consegue acessar o prontuário de um paciente que só é atendido pelo médico B.
- Teste de regressão: todos os testes já existentes do roadmap 1.0 (Fase 11) continuam passando sem alteração.
- Teste de migration: subir o banco do zero (Testcontainers) e confirmar que todas as migrations, antigas e novas, rodam em sequência sem erro.

✓ Definição de pronto (DoD) desta fase

- `mvn test` roda toda a suíte (antiga + nova) sem falhas
- Existe cobertura de teste para cada célula "NÃO" da matriz de permissões deste documento
- O teste de posse de registro (médico A x paciente do médico B) está implementado e passando

2.12 Documentação OpenAPI com segurança

Fechando o ciclo: documentar o que agora é, de fato, um sistema com controle de acesso real.

O que fazer

- Configurar o `securityScheme` Bearer JWT no springdoc-openapi (se ainda não configurado), para que o Swagger UI permita autenticar e testar rotas protegidas diretamente pela interface.
- Documentar, na descrição de cada endpoint sensível, quais roles têm acesso (ex.: via `@Operation(description = "Acesso: MEDICO (próprios pacientes), ADMIN")`).
- Adicionar uma seção no `README.md` explicando a matriz de permissões deste documento, para qualquer pessoa nova no projeto entender rapidamente o modelo de acesso.

✓ Definição de pronto (DoD) desta fase

- O Swagger UI permite autenticar com um token JWT e testar rotas de diferentes papéis
- Cada endpoint sensível tem, na sua documentação, quais roles podem acessá-lo
- `README.md` atualizado com a matriz de permissões e o guia de criação de usuários por papel

Anexo A — Prompt pronto para entregar à IA

Copie, adapte o número da fase, e cole numa IA de codificação que tenha acesso ao seu repositório atual.

Modelo de prompt — Roadmap 2.0

Você vai me ajudar a implementar a Fase [NÚMERO] do Roadmap 2.0 do MedFlow. Este NÃO é um projeto novo – é uma expansão sobre um sistema já funcionando. Leia com atenção o que já existe antes de gerar qualquer código.

Contexto do que já existe (não pode quebrar):

- Backend: Java 21, Spring Boot, Spring Security + JWT, JPA/Hibernate, Flyway, PostgreSQL, RabbitMQ
- Roles atuais: ADMIN, MEDICO, PACIENTE
- Entidades atuais: Usuario, Medico, Paciente, Especialidade, Consulta, Prontuario, Exame, Pagamento
- Frontend: React + Vite + TypeScript + Tailwind + React Router + TanStack Query, com AuthContext e Layout condicional por role já implementados
- Infra: Docker, Docker Compose, Kubernetes, CI/CD já configurados

Objetivo desta fase (Roadmap 2.0):

[cole aqui o conteúdo da fase correspondente deste documento: objetivo, o que fazer, entregáveis]

Matriz de permissões que esta fase precisa respeitar (não altere sem me avisar):

[cole aqui a tabela "Matriz de acesso por recurso" deste documento, ou a(s) linha(s) relevante(s) para esta fase]

Definição de pronto (DoD) que preciso validar ao final:

[cole aqui os itens do "Definição de pronto" desta fase]

Regras obrigatórias:

1. Toda mudança de schema deve vir como uma NOVA migration Flyway, nunca alterando uma migration já existente.
2. Nenhum endpoint, rota ou comportamento já existente pode ser removido ou ter seu contrato alterado, exceto se esta fase disser explicitamente o contrário.
3. Toda regra de permissão deve ser garantida no BACKEND (Spring Security / @PreAuthorize), nunca só escondida no frontend.
4. Implemente apenas o escopo desta fase – não adiante fases futuras deste roadmap.
5. Se algo estiver ambíguo ou conflitar com o que já existe no código atual, pergunte antes de assumir.
6. OBRIGATÓRIO – siga à risca o "Anexo C – Protocolo de Testes Rigorosos" deste documento: você não pode declarar esta fase concluída sem antes escrever e RODAR de verdade uma suíte de testes cobrindo cenário feliz, cenário de erro, segurança/permissão, concorrência (quando aplicável) e regressão das fases anteriores. Cole o output real da execução dos testes (não um resumo) na sua resposta final, com a contagem de testes passando/falhando. Se algum teste falhar, corrija o código e rode de novo antes de me entregar a fase – eu não devo ser eu quem encontra o primeiro bug.

Dica extra

Antes de colar o prompt, garanta que a IA tem acesso ao código atual do repositório (via upload, acesso ao Git, ou colando os arquivos relevantes) — sem isso, ela vai "adivinhar" a estrutura existente e o risco de quebrar algo aumenta muito.

Anexo B — Checklist geral de entrega

Progresso do Roadmap 2.0, fase a fase.

- ✓ Fase 2.1 — Novos papéis criados, compatibilidade com usuários existentes garantida
- ✓ Fase 2.2 — Médicos com múltiplas especialidades funcionando
- ✓ Fase 2.3 — Dados cadastrais separados de dados clínicos, a nível de DTO e permissão
- ✓ Fase 2.4 — Method Security implementado, com checagem de posse de registro
- ✓ Fase 2.5 — Módulo de triagem funcionando, com acesso restrito correto
- ✓ Fase 2.6 — Módulo de recepção funcionando, com acesso restrito correto
- ✓ Fase 2.7 — Prontuário, prescrições e cirurgias protegidos e funcionando
- ✓ Fase 2.8 — Módulo de faturamento funcionando, sem vazamento de dado clínico
- ✓ Fase 2.9 — Auditoria registrando acessos e alterações sensíveis
- ✓ Fase 2.10 — Frontend com páginas e proteção de rotas para todos os papéis
- ✓ Fase 2.11 — Suíte de testes de segurança cobrindo toda a matriz de permissões
- ✓ Fase 2.12 — Documentação OpenAPI e README atualizados

✓ Checagem final de compatibilidade

Antes de considerar o Roadmap 2.0 concluído, confirme que: nenhum paciente, médico ou admin já cadastrado no MedFlow 1.0 perdeu acesso ou teve comportamento alterado; todos os testes do roadmap 1.0 continuam passando; e o sistema sobe normalmente via `docker compose up` e via Kubernetes, exatamente como antes, só que agora com os novos módulos disponíveis.

Anexo C — Protocolo de testes rigorosos obrigatório

A regra mais importante para você não virar o "testador manual" da IA: nenhuma fase é considerada pronta sem uma bateria de testes real, executada e comprovada.

⚠ O problema que este anexo resolve

Sem uma regra explícita, é muito comum a IA dizer "pronto, deve funcionar" e empurrar pra você o trabalho de testar manualmente, achar o bug, voltar e pedir correção — um ciclo lento e frustrante. Este anexo transforma isso numa **exigência de entrega**: a IA só pode dizer que uma fase está concluída depois de ela mesma ter rodado testes de verdade e colado o resultado.

Princípio geral

Para toda fase deste roadmap (1.0 e 2.0), antes de declarar a fase concluída, a IA deve escrever e **executar** — não apenas descrever — uma suíte de testes automatizados que simule o uso real do sistema, incluindo tentativas de *quebrar* a regra de negócio implementada. O objetivo não é só "o caminho feliz funciona", é "o sistema se comporta corretamente mesmo quando alguém tenta usá-lo errado, sem permissão, ou ao mesmo tempo que outra pessoa".

As 5 categorias de teste obrigatórias por fase

Categoria	O que verifica	Quando se aplica
1. Caminho feliz	A funcionalidade funciona corretamente com dados válidos e usuário com permissão correta	Toda fase
2. Caminho de erro / validação	Dados inválidos, campos obrigatórios ausentes, IDs inexistentes retornam erro claro (400/404), sem quebrar o servidor	Toda fase que recebe input do usuário
3. Segurança / permissão	Usuário sem autenticação recebe 401; usuário autenticado sem permissão recebe 403; usuário tentando acessar registro de outra pessoa (ex.: prontuário alheio) é bloqueado	Toda fase com rota protegida (praticamente todas a partir da Fase 4 do roadmap 1.0)
4. Concorrência / condição de corrida	Duas requisições simultâneas competindo pelo mesmo recurso (ex.: dois agendamentos no mesmo horário do mesmo médico) resultam em exatamente um sucesso e um erro controlado — nunca os dois passando	Fases com regra de exclusividade: agendamento de consultas, cirurgias, confirmação de pagamento
5. Regressão	Toda a suíte de testes das fases anteriores continua passando sem nenhuma alteração de comportamento não intencional	Toda fase, sem exceção

Protocolo de execução — passo a passo que a IA deve seguir

1. Implementar o código da fase, conforme o escopo definido.
2. Escrever os testes das 5 categorias acima que se apliquem àquela fase, usando JUnit 5 + Mockito (unitários) e Testcontainers (integração com banco real em container, nunca H2 em memória).
3. Rodar `mvn test` (backend) e, quando houver mudança de frontend, os testes correspondentes (ex.: Vitest/React Testing Library).
4. Se algum teste falhar: corrigir o código (não o teste, a menos que o teste estivesse de fato errado) e rodar novamente — repetir até a suíte inteira ficar verde.
5. Rodar também a suíte completa do projeto (não só os testes novos), para garantir que nada de fases anteriores quebrou.
6. Colar, na resposta final da fase, o **output real do terminal** da execução dos testes (contagem de testes, tempo, "BUILD SUCCESS"), não um resumo em prosa como "os testes passaram".

7. Só então apresentar a fase como concluída, junto com a lista de comandos para você reproduzir os testes localmente, se quiser conferir.

💡 Frase para cobrar isso da IA, se ela pular a etapa

Se em algum momento a IA disser algo como "implementado, deve funcionar" sem colar output de teste, responda: "Antes de seguir, rode a suíte de testes desta fase (as 5 categorias do Anexo C) e cole o output real da execução. Não avance sem isso."

Modelo de simulação de concorrência (quando aplicável)

Para regras de exclusividade (o caso mais clássico sendo o conflito de agendamento), a IA deve simular concorrência de verdade, não apenas testar sequencialmente. Um teste de exemplo aceitável:

```
@Test
void duasRequisicoesSimultaneasParaOMesmoHorario_apenasUmaDeveSerAceita() throws Exception {
    ExecutorService executor = Executors.newFixedThreadPool(2);
    CountDownLatch largada = new CountDownLatch(1);

    Callable<Integer> tentativa = () -> {
        largada.await(); // as duas threads esperam o mesmo sinal de largada
        return tentarAgendarConsulta(medicoId, mesmoHorario).getStatusCode();
    };

    Future<Integer> resultado1 = executor.submit(tentativa);
    Future<Integer> resultado2 = executor.submit(tentativa);
    largada.countDown(); // libera as duas ao mesmo tempo

    List<Integer> statusCodes = List.of(resultado1.get(), resultado2.get());
    assertTrue(statusCodes.contains(201)); // uma teve sucesso
    assertTrue(statusCodes.contains(409)); // a outra recebeu conflito, não quebrou o servidor
}
```

Modelo de tabela de cenários — a IA deve entregar algo assim a cada fase

Além do código de teste, a IA deve resumir os cenários cobertos numa tabela simples junto da entrega, assim:

Cenário	Categoria	Resultado esperado	Status
Recepcionista agenda consulta para paciente existente	Caminho feliz	201 Created	✓ Passou
Agendar consulta com data no passado	Validação	400 Bad Request	✓ Passou
Triagem tenta acessar prontuário clínico	Segurança	403 Forbidden	✓ Passou
Duas requisições simultâneas, mesmo horário, mesmo médico	Concorrência	1x 201, 1x 409	✓ Passou
Suíte completa de fases anteriores	Regressão	Todos os testes antigos continuam verdes	✓ Passou

Checklist específico por tipo de fase

Além das 5 categorias gerais, alguns tipos de fase pedem verificações extras específicas:

Tipo de fase	Verificação extra obrigatória
Fases com migration de banco (ex.: 2.1, 2.2)	Subir o banco do zero (container novo) e confirmar que todas as migrations — antigas e novas — rodam em sequência sem erro, e que dados de exemplo pré-existentes (seed) permanecem íntegros após a migração
Fases de autenticação/autorização (ex.: 4, 2.4)	Testar explicitamente: sem token, token expirado, token de role errada, e tentativa de acessar recurso de outro usuário trocando o ID manualmente na URL
Fases de regra de negócio com estado (ex.: 7, 2.7 cirurgias)	Testar transições de estado inválidas (ex.: tentar "realizar" uma consulta já cancelada) e confirmar que são bloqueadas com erro claro
Fases de frontend (ex.: 13, 2.10)	Teste automatizado (ex.: Vitest/Testing Library) garantindo que um componente/rota protegida não renderiza conteúdo sensível para um role sem permissão, mesmo que a chamada de API falhe
Fases de containerização/infra (ex.: 14, 16)	Ciclo completo "derrubar e levantar": <code>docker compose down -v && docker compose up -d --build</code> (ou equivalente Kubernetes) executado do zero, confirmando que o sistema volta a funcionar sem intervenção manual

Resumo da regra, em uma frase

Nenhuma fase é "entregue" — é apenas "implementada" — até que a própria IA tenha rodado a suíte de testes completa, colado o resultado real, e confirmado que todas as 5 categorias relevantes àquela fase estão verdes.